

Московский государственный университет имени М.В. Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра алгоритмических языков

Коробов Евгений Алексеевич

Интерпретатор диалекта языка Лисп

Практическое задание

Преподаватели:

Шамаева Елена Денисовна
Елисеева Мария Алексеевна
Уразов Станислав Олегович

Москва, 2026

Содержание

1. Постановка задачи 3
2. Теоретическая часть 3
3. Описание программы 4
3.1 Модули 4
3.2 Структуры данных и состояние интерпретатора 5
3.3 Встроенные функции 5
3.4 REPL и обработка файлов 6
4. Используемые библиотеки 6
5. Пример работы программы 7
5.1 Основной интерфейс 7
5.2 Работа программы 7
5.3 Обработка ошибочных ситуаций 8
6. Использование нейросетевых средств 8
7. Выводы 9
8. Литература 10

1. Постановка задачи

Цель работы — разработать на языке Haskell интерпретатор диалекта языка программирования Лисп, включающего функциональное ядро и расширяющего понимание S-выражения. Требуется реализовать:

- поддержку атомов (символов, чисел, строк, пустого списка nil или ());
- базовые функции Лиспа: car, cdr, cons, eq, atom, quote, eval, cond;
- некоторые дополнительные встроенные функции: предикаты (numberp, atomp, listp, =), функции работы со списками (append, list, member), арифметические операции (+, -, *, /, >, <, >=, <=), логические (and, or, not), предикаты null, eql, функции caar, cadr, cddr, cdddr, cadar;
- средства определения пользовательских функций (defun) с аппликативным порядком вычисления;
- парсер, строящий абстрактное синтаксическое дерево (AST);
- модуль вычисления с простой интерактивной средой (REPL).

Программа должна корректно обрабатывать синтаксические и семантические ошибки и предоставлять возможность выполнять программы из файлов.

2. Теоретическая часть

Реализуемый диалект Лиспа базируется на классическом понятии *S-выражения*. *S-выражением* может быть:

- **атом** – символ (например, *x, append*), число (целое или с плавающей точкой), строка ("*hello*"), логическое значение T, пустой список *nil* или *()*;
- **точечная пара** – запись вида (A . B), где A и B – произвольные S-выражения (обычно используется для представления списков, но в общем случае позволяет строить произвольные бинарные деревья);
- **список** – частный случай точечной пары, где последний элемент – *nil* (например, *(a b c)* эквивалентно *(a . (b . (c . nil)))).*

Порядок вычисления – **аппликативный** (строгий): сначала вычисляются все аргументы функции, затем полученные значения подставляются в тело функции. Это соответствует правилу:

(f a1 a2 ...) → вычислить a1, a2, … до значений, затем применить f к ним.

Исключения составляют специальные формы:

- **quote** – не вычисляет свой аргумент, а возвращает его как данные;
- **cond** – вычисляет предикаты последовательно до первого истинного;
- **defun** – не вычисляет тело при определении, а сохраняет его для последующего вызова.

Для организации вычислений интерпретатор проходит следующие этапы:

1. **Лексический анализ** – разбиение входной строки на *поток токенов* (лексем): числа, символы, операторы, скобки и т.д.
2. **Синтаксический анализ** – построение *абстрактного синтаксического дерева* (AST) из потока токенов. AST в данной реализации представлено типом ExprTree SExpr.
3. **Семантический анализ** – выделение *определений функций* (defun) и проверка корректности (например, отсутствие двойных определений).
4. **Вычисление** – обход AST в среде определений с применением аппликативного порядка.

3. Описание программы

3.1 Модули

Проект состоит из следующих файлов:

- **Types.hs** хранит основные типы данных: лексемы, AST дерево, описание пользовательской функции
- **Lexer.hs** – лексический анализатор: преобразование строки в список токенов
- **Parser.hs** – синтаксический анализатор: строит AST дерево из потока токенов
- **Semantic.hs** – семантический анализ: выделяет *defun*, проверяет наличие повторных определений
- **Interpreter.hs** – основной модуль вычисления: выполняет программу AST дерева. Реализует встроенные функции

- **Utils.hs** – тут содержаться вспомогательные функции: сравнение деревьев, работа с числами, подстановка параметров, отображение и т.д.
- **Main.hs** запускает REPL или обработку файлов

3.2 Структуры данных и состояние интерпретатора

- **LexType** – тип лексемы (скобки, числа, символы, арифметические операции и т.д.).
- **Lex** – токен с номером строки, позицией и типом.
- **SExpr** – S-выражение: атом (символ/число/строка) *SAtom,* пустой список *SNil,* список *SList*.
- **ExprTree a** – абстрактное синтаксическое дерево с значениями в листовых вершинах: *Leaf a* или *Node [ExprTree a]*. В интерпретаторе используется *ExprTree SExpr*.
- **Definition** – определение верхнего уровня: либо *Fun UserFun* (пользовательская функция), либо *Expr (ExprTree SExpr)* (выражение для вычисления).
- **UserFun** – запись пользовательской функции: имя, список параметров, тело (список выражений).

Состояние интерпретатора в REPL – это список определений [Definition], который накапливается по мере ввода новых выражений и определений.

3.3 Встроенные функции

Встроенные функции реализованы как отдельные обработчики:

- **car, cdr** –возвращают голову и хвост списка
- **cons** – конструирует список
- **eq, eql** – поверхностное сравнение атомов
- **atom** – проверка, является ли аргумент атомом (не списком)
- **quote** – возвращает аргумент без вычисления
- **eval** – форсирование вычисления переданного списка
- **+, -, *, /** – арифметика с целыми и вещественными числами
- **=, >, <, >=, <=** – сравнение чисел
- **and, or, not** – логические операции (короткое замыкание для *and/or*)
- **null** – проверка на пустой список

- **append** – слияние двух списков
- **member** – проверка вхождения элемента в список
- **intern** – конкатенация атомов
- **caar, cadr, cddr, cdddr, cadar** – комбинации *car* и *cdr*
- **print** – вывод значения на экран

Пользовательские функции (**defun**):

- При вызове происходит связывание параметров с вычисленными аргументами.
- Затем тело вычисляется в текущем окружении определений (пользовательская функция видит все ранее определённые функции, включая себя для рекурсии).

3.4 REPL и обработка файлов

REPL: при запуске без аргументов входит в интерактивный режим.

- Приглашение >>>;
- Команда **exit** завершает работу;
- Каждая введённая строка лексически и синтаксически анализируется, затем семантически, после чего выполняются вычисления через *run*. Результат вычисленных выражений выводится в консоль;
- Определения функций накапливаются и доступны в последующих вводах;

Обработка файла: если передан аргумент *filename*, программа читает файл, выполняет все выражения и сообщает об успехе или ошибке.

Обработка ошибок: все этапы (лексика, синтаксис, семантика, вычисление) возвращают Either String a, ошибки выводятся пользователю.

4. Используемые библиотеки

Проект использует только стандартную библиотеку Haskell и встроенные модули:

- **Data.Char** – для классификации символов (цифры, буквы, пробелы);
- **Data.List** – для работы со списками;

- **System.Environment** – для получения аргументов командной строки;
- **System.IO** – для работы с файлами и буферизацией вывода.

5. Пример работы программы

5.1 Основной интерфейс

При запуске без аргументов открывается REPL:

```
---Interpretation mode---
Enter program (for end enter 'exit'):
------------------------
>>>
```

Рис. 1: Интерфейс REPL

Пользователь вводит построчно выражения. Каждое выражение вычисляется, результат (если это не *defun*) выводится на экран.

5.2 Работа программы

Пример сессии:

```
---Interpretation mode---
Enter program (for end enter 'exit'):
------------------------
>>>(print (+ 1 2))
3
>>>(print (append '(a b) '(c d)))
(A B C D)
>>>(defun square (x) (* x x))
>>>(print (square 5))
25
>>>(print (cond ((> 3 4) 10) (T 20)))
20
>>>(print (car (quote (hello world))))
HELLO
>>>exit
End of interpretation
```

Рис. 2: Пример интерактивной работы

```
PS C:\Users\Jenya\Desktop\haskel\jeka> stack exec jeka-exe test1.lsp
"abs"
4
(CDR (' (B CAR (nil 10))))
"G"
(-1 CAR (nil 10))
"G"
(B CAR (nil 10))
Compilation is successful
```

Рис. 3: Пример интерпретации файла

5.3 Обработка ошибочных ситуаций

Интерпретатор выдаёт понятные сообщения об ошибках:

```
>>>print (g '() (-1)))
Parser error: Syntax error: Expected '(' at 0::0
>>>(print (g '() (-1)))
Interpreter error: Not find function 'G' at 0::8
>>>(defun f (x y) (+ x y))
>>>(f 5)
Interpreter error: Wrong number of arguments: 'F' is expected to contain 2 argument.
>>>
```

Рис. 4: Обработка ошибок

6. Использование нейросетевых средств

При разработке интерпретатора нейросетевые модели использовались как интерактивный справочник по синтаксису Haskell и принципам построения парсеров.

В частности, были заданы следующие вопросы:

- «Как в Haskell представить S-выражение с точечными парами в виде алгебраического типа?» – получен совет использовать рекурсивный тип SExpr.
- «Укажи основные отличия в организации вычислений функциональных выражений в лисп и хаскель» – нейросеть объяснила различия, привела пример правильного порядка вычислений для языка Lisp.
- «Как реализовать аппликативный порядок вычисления для пользовательских функций?» – нейросеть объяснила необходимость вычисления аргументов до подстановки и привела пример с replaceLeafs.

- «Является ли данная грамматика допустимой для рекурсивного спуска?» – нейросеть немного исправила грамматику и объяснила, почему новая грамматика допустима.

Кроме того, нейросеть помогла отформатировать код, убрать warnings. Благодаря этому удалось ускорить разработку.

7. Выводы

В результате выполнения практического задания разработан полноценный интерпретатор диалекта Лиспа на языке Haskell.

Достигнуты следующие результаты:

- Реализованы лексический, синтаксический и семантический анализаторы;
- Построено абстрактное синтаксическое дерево (AST);
- Реализован аппликативный порядок вычисления;
- Поддержаны базовые и дополнительные встроенные функции (всего более 20);
- Работают пользовательские функции (*defun*) с подстановкой параметров;
- Обеспечена интерактивная среда REPL с накоплением определений;
- Предусмотрена обработка синтаксических и семантических ошибок.

Проект демонстрирует преимущества функционального подхода: алгебраические типы данных идеально подходят для представления S-выражений, рекурсия позволяет естественно реализовать парсер и интерпретатор, а система типов Haskell обеспечивает возможности отладочных средств для интерпретатора.

8. Литература

1. The Haskell 2010 Report – официальное описание языка Haskell. Url: https://www.haskell.org/onlinereport/haskell2010/
2. Common Lisp the Language, 2nd Edition – Guy L. Steele Jr. (описание S-выражений и основных функций). Url: https://filonenko-mikhail.github.io/cltl2-doc/enpdf/cltl2.pdf
3. Write Yourself a Scheme in 48 Hours – пошагово описывает создание интерпретатора Scheme на Haskell. Url: https://upload.wikimedia.org/wikipedia/commons/a/aa/Write_Yourself_a_Scheme_in_48_Hours.pdf?utm_source=en.wikibooks.org&utm_campaign=index&utm_content=original
4. Interpreting LISP – (Gary D. Knott), в книге разбираются структуры данных и процесс создания работающего Lisp-интерпретатора. Url: https://www.civilized.com/files/lispbook.pdf